Module 2 — Workflows & Best Practices

Lesson 2.5

Git, commits, and PRs

Let Claude write commits and PRs, but enforce hygiene: small commits, descriptive messages, never amend published work.

Why it matters

Claude can produce excellent commit messages and PR descriptions — they're synthesis tasks, which it's good at. But it can also lose work via destructive git operations if you don't set guardrails. This lesson is the safe baseline.

The safe-by-default rules

These should hold in every Claude-driven git workflow:

1. **Never amend a commit you've pushed.** Always make a new commit.
2. **Never force-push to a shared branch** (`main`, `master`, `develop`, or a teammate's feature branch).
3. **Never skip hooks** (`--no-verify`) without a stated, accepted reason.
4. **Stage files explicitly by name** rather than `git add -A` or `git add .` — avoids accidental secret or build-artifact commits.
5. **Investigate before destroying.** Unfamiliar files or branches may be in-progress work — read before deleting.

Claude Code's default prompts and built-in skills reinforce these. Don't undo them with `--dangerously-skip-permissions` outside a sandbox.

Commit hygiene

A good commit has a clear scope and a clear message:

```
fix(orders): prevent duplicate insert under concurrent idempotency-key writes

Wrap the existence check and insert in a single SELECT ... FOR UPDATE
transaction. Previously two concurrent requests could both pass the
check and then both insert, raising IntegrityError.

Repro and regression test added.
```

The structure: type(scope) terse summary, blank line, body explaining *why*. Claude can write this — just give it the scope:

> *"Commit the staged changes. Type is `fix`, scope `orders`. Explain the root cause in the body. Reference the test we just added."*

Asking Claude to commit

The default protocol Claude Code follows:

1. Runs `git status` to see what's changed
2. Runs `git diff` to see the changes
3. Reads recent `git log` to match the project's commit-message style
4. Drafts a message
5. Stages files by name
6. Creates the commit
7. Confirms with `git status`

You can short-circuit any step if you trust the state ("the diff is already what I want, just commit with message X").

Do not run `git commit` without reading the diff first. Claude does this, you should too. Surprises in diffs are the cheapest bug to catch.

PRs

`gh pr create` is the right tool. Claude can run it:

```
"Create a PR. Title: 'fix(orders): prevent idempotency-key races'.
Body: include a Summary section and a Test plan section.
Base branch: main."
```

Good PR descriptions are short:

```
## Summary
- Wraps existence check + insert in a transaction with row-level locking.
- Adds regression test reproducing the original race.

## Test plan
- [x] `pytest tests/test_orders.py::test_concurrent_idempotency`
- [x] Manual: run the load test in `bench/orders_load.py`, observe zero IntegrityErrors
```

Claude is good at producing this format. Avoid letting it write a five-paragraph essay nobody will read.

What NOT to let Claude do without confirmation

- `git push --force` (especially to a shared branch)
- `git reset --hard`
- `git branch -D <name>`
- `git checkout -- .`
- Merging or rebasing onto `main`
- Closing or merging a PR
- `gh pr merge`

Each of these is reversible-but-expensive at best, irreversible at worst. Claude should ask before doing them.

Workflow patterns

Stacked commits during a long task

Don't accumulate 800 lines of uncommitted changes. After each successful step:

> *"Stage and commit just `src/foo.py` and the related tests. Type `feat`, scope `foo`. Body: 'add the X capability'."*

Small commits let you revert one thing at a time without losing unrelated work.

Cleaning up before PR

> *"Walk through* `git log origin/main..HEAD`*. Are there any commits that should be squashed or reworded for clarity? Suggest a rebase plan but don't execute it."*

You stay in control of the rewrite.

Try it

1. Make a small change, ask Claude to commit it (don't tell it the style — see what it picks up from `git log`).
2. Make a second small change in a new branch, push it, then ask Claude to open a PR with a clear test plan.
3. Try asking Claude to force-push (don't actually do it) — note how it confirms first.

Gotchas

- **`git add -A` after a long agent session** can sweep in artifacts (`.pyc`, `.DS_Store`, generated files). Always look at `git status` first.
- **Commit messages with no body** are a smell on non-trivial changes. Insist on a body that explains *why*.
- **PRs with no test plan** are a smell. Even if the test plan is just one line, it forces you to articulate verification.
- **Claude will sometimes try `git commit --amend`** to fix a typo'd previous message. On unpushed commits that's fine; on pushed commits it's destructive. Set the rule, hold the line.

Further reading

- 2.3 Review and refactor — use `/review` before pushing
- Built-in `/security-review` for changes that touch auth, crypto, or secrets

Next

→ 2.6 When to delegate to a subagent